コンピュータの言語2025：課題5：フローチャートとPython (while)

前回までに `for` の使い方を学びましたが，今回は `while` を使った繰り返しです．

課題ファイルの中のプログラムは全て実行するようにしてください．

while を使った繰り返し

`while` は

```
while 条件:
    命令
```

のように書き，条件が満たされている間，字下げをされた部分が繰り返されます．試してみましょう．

In [ ]:
```
n = 0
while n<10:
    print(n)
    n = n+1
```

上記のように，while ループの中で「nが10未満」という条件が成り立っている間，「nに1を足す」命令が繰り返されています．

例１：選択構造による繰り返し（前判定）

In [ ]:
```
summ = 0
i = 1

while i<=10:
    summ = summ + i
    i = i+1

print(summ)
```

参考：なぜ　`sum` を使わないのか？

`sum` は python の組み込み関数の名前として使われていますので，`sum` を変数として使うと組み込み関数が上書きされています．

In [2]:
```
sum([1,2,3])  #組み込み関数として利用．1+2+3を計算する．
```

Out[2]:
```
6
```

In [3]:
```
sum = 3     #変数として利用.
print(sum)
```

```
3
```

In [4]:
```
sum([1,2,3]) # sum は上書きされてしまったのでエラーになる
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 sum([1,2,3])

TypeError: 'int' object is not callable
```

このように，一度変数として使ってしまうと，上書きされて関数として利用できなくなってしまいます． 組み込み関数として復活させたいときには，python 内の変数としての `sum` を `del` 命令で削除します．

In [5]:
```
del sum        #変数（オブジェクト）sum を削除
sum([1,2,3])   #組み込み関数として復活！
```

Out[5]:
```
6
```

練習１

- 下のプログラムは1から10までの和を求めるプログラムです．このプログラムを改変して１０から３００までの和を求めてください．（１０と３００は含みます）

In [ ]:
```
summ = 0
i = 1

while i<=10:
    summ = summ + i
    i = i+1

print(summ)
```

例２：選択構造による繰り返し（後判定）

while は「条件が成立するか成立しないか」，つまり「条件が `True` か `False` か」でループを回すかどうかを決定します．（このように `True` または `False` になる式を **論理式** といいます．）

while の条件のところを `True` としておいて，ループの中で実行条件や終了条件を判断することもできます．

In [ ]:
```
summ = 0
i = 1

while True:
    summ = summ + i
    i = i+1

    if i<=10:
        continue  # whileループの先頭に戻る
    else:
        break     # whileループを終了する

print(summ)
```

`continue` が実行されると，while ループの最初に戻ります（その後の命令は実行されません）．また，`break` が実行されると，while ループはそこで終了し，プログラムはループを抜けて while の下の命令が実行されます.

練習2

`continue` あるいは `break` の前後に命令を書いて実行されるかどうかを調べてみましょう.

In [ ]:
```
summ = 0
i = 1

while True:
    summ = summ + i
    i = i+1

    if i<=10:
        #ここに書いた命令は何回実行される？->(a)
        xxxxxxxxx
        continue
    else:
        #ここに書いた命令は何回実行される？->(b)
        yyyyyyyyy
        break

    print("hello")  # hello  は何回出力される？ ->(c)

print(summ)
```

(a): XXX 回, (b): YYY 回, (c) ZZZ 回

例3

このフローチャートに合うように以下のプログラムの XXXX を埋めてください.

In [ ]:
```
i = 7

while i>0:
    XXXXXXXX
    XXXXXXXX

print("owari")
```

スライドでは，コンピュータを使わずに考える問題でしたが，コンピュータでプログラムを実行して，実行回数を調べてみましょう． そのために何回実行されるかを数えるための変数 "repeat" を用意して， `repeat` は繰り返されるたびに 1 が足されるようにし，その値を出力するようにします．

In [ ]:
```
i = 7
repeat = 0 #繰り返し回数を調べるための変数
while i>0:
    repeat = repeat + 1
    XXXXXXXXXX
    XXXXXXXXXX

print(f'繰り返しは{repeat}回でした')
```

このようにすることでループの繰り返し回数を確認することができます．

このように，プログラムの動作を確認したいときには，自分で変数を用意しどのような値が出力されるのかを確認するとよいでしょう．

練習3

上記の問題に対応する Python プログラムは以下となります．**xx(1)xx** と **xx(2)xx** に入る式は何になるでしょうか？（ヒント：sum = 45 と出力されれば合っていることになります）

In [ ]:
```
summ = 0
xx(1)xx

while i<=13:
    summ = summ + i
    xx(2)xx

print(f'sum={summ}')
```

練習4

あるウィルスの感染者数は 1 日あたり2.3%ずつ増えていくことがわかっているとします．ある日の感染者は100人でした．感染者が100,000人を超えるのは約何日後になるでしょうか？それを求めるためのプログラムを完成させてください．

In [ ]:
```
n = 100
days = 1
while xx(1)xx:
    days = days+1
    n = xx(2)xx

print(f'{days}日')
```

ループ端記号による繰り返し

フローチャートでは，繰り返しの処理を行うのに選択記号もループ端記号も使えますが，それは for, while に対応しているわけではありません．今回はループ端記号のフローチャートを while を使ってプログラムにしていきます．

ループ端記号に書かれている条件は「**ループが終了する条件**」であるのに対し，プログラムの while のところに書く条件は「**ループが実行される条件**」であるということに注意してください．

In [ ]:
```
n = int( input("nを入力してください:") )

while n>0:
    print(f'n={n}')
    n = n-1

print("owari")
```

無限ループ

上のプログラムで n<0 として -1 を入力すると，n はどんどん小さくなって，いつまでもループが終了しなくなってしまいます．このように終了しないループのことを **無限ループ** といいます（場合によっては操作不能になるので，やめておいたほうがよいでしょう）．もしこのようになったプログラムを Google Colab で終了させるためには，メニューの 「 **ランタイム** 」の中から「 **実行を中断** 」または「 **ランタイムを再起動** 」を選択してください．運が良ければ再開することができます．

練習5

上のフローチャートと同じになるように下のプログラムを完成させてください．

In [ ]:
```
max_val = 0
while True:
    n = int(input("数字を入力してください："))
    if XXXXXXXX:
        break
    if XXXXXXXX:
        max_val = n
print( "最大値：", max_val)
```

練習6

上記のプログラムは終了条件と実行条件を入れ替えて，下のように書くこともできます．

これと同じになるように下のプログラムを完成させてください．

In [ ]:
```
max_val = 0
n = 1
while n>0:
    n = int(input("数字を入力してください："))
    if XXXXXXXX:
        max_val = n
print( f"最大値：{max_val}")
```

関数

関数とは，**一連の処理をひとまとめにしたもの**，あるいは，**処理を行った後にある値を返すもの**　です．

関数は以下のように定義します．

```
def 関数名（パラメータ):
    処理ブロック
```

while, for と同様に，処理ブロックは４つのスペースで字下げされている部分になります．

パラメータ の部分は関数が値を受け取るところで，　**引数** といいます．

関数が変数の値を返す場合には `return` によって返します（必ずしも値を返す必要はありません．その場合は `return` は不要です）．

```
def 関数名（パラメータ):
    処理ブロック
    return 変数
```

例えば，「ある値を受け取って１を加えて返す」関数 ***plus*** は以下のように書くことができます．

In [ ]:
```
def plus(x):
    x = x+1
    return x

plus(1)
```

注意点としては，**関数を使う時には，必ずその前に定義されていなければならない**ということです．

以下はエラーが出るプログラムの例

In [ ]:
```
print( minus(1) )    # 定義されていないのに，いきなり関数 minus が現れた！　エラー

def minus(x):
    return x+1
```

正しい順序の例．

In [ ]:
```
def minus(x):        # ここで定義されている
    return x-1

print( minus(1) )    # 上で定義されているので，使える！
```

（注意）google colab の場合，一度実行されたセルの変数や関数は全て保存されていますので，上の方のセルで変数や関数の定義を実行していれば，後のセルでどこでも利用できます．

練習7

下のプログラムで，引数として入力された値に3をかけて返す関数 kakesan を定義してください．

In [ ]:
```
def xxxxxxxxxxxx:
    xxxxxxxx
    xxxxxxxx


x = kakesan(3)
print(x)
```

複数の引数

関数は複数の引数を取ることができます．

In [ ]:
```
def select_small(x,y):
    if x<y:
        return x
    else:
        return y

p = select_small(1,3)
print(p)
```

複数の値の return

関数は複数の値を返すことができます.

In [ ]:
```
def tasu_1_2(a):
  x = a+1
  y = a+2
  return x,y

p,q = tasu_1_2(3)
print(p,q)
```

練習8

２つの数字を引数として受け取り，その掛け算を出力する関数kake を定義してください.

In [ ]:
```
def xxxxxxxxxxxxxxxx:
        xxxxxxxxx


x = kake(2,3)
print(x)
```

練習9

1つの数字を引数として受け取り，その数を 3 で割った商とあまりを返す関数 warusan を定義してください.

In [ ]:
```
def xxxxxxxxxxxxxxxxx:
    shou = xxxxxxx
    amari = xxxxxx
    return xxxxxxxxxxxxxxx

a, b = warusan(10)
print(f'10を3で割った商は{xxx},あまりは{xxx}です')
```

練習10

4次関数 $y =x^4-2x^3+1$ について，x を -1 から 2 まで 0.01 ずつ動かした時の y の最小値とそのときの x の値を求めるプログラムを書いてみましょう．以下のプログラムの **XXXXXXXXXXXXXXXX** には何を入れたらよいでしょうか？

In [ ]:
```
def func(x):
    return xxxxxxxxxxxxxxxx

x = -1.0
min_val = func(x)
min_x   = x
while x<=2.0:
    y = func(x)
    if XXXXXXXXXXXX:
        min_val = XXXXXXXXXXXXXX
        min_x   = x
    x = XXXXXXXXXXX

print(f"最小値は{min_val}，その時のxの値は{min_x}"
```

出力された最小値とその時の x の値が合っているか？は以下のプログラムを実行してグラフを見て判断してください．

In [ ]:
```
# プログラムの意味は理解する必要はありません
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-1.0,2.0,100)
y = x**4 - 2*(x**3) + 1

plt.plot(x,y)
```

関数とスコープ (応用)

基本的には，関数は引数や返り値を通して周囲とやりとりをしますし，多くの場合ではそのようにすべきです．こうすることで，情報の入力と出力がはっきりして，読みやすくなり，テストやデバッグがやりやすくなります．

これはプログラムを理解しようとするときに，**その場所の近辺さえ理解しさえすれば理解可能である** ということです．もし他の場所で定義されたものまで考えなければならないとしたら，その関数が実行される文脈など多くのことを考慮しなければならず，理解が難しくなってしまいます．

しかしながら，それでも，ときには，**関数の外で定義した変数（広域変数（グローバル変数））にアクセスしたい** 場合もあります．そのときには，どうすればよいのでしょうか？ そんなときは `global` を使います．

練習11

下のプログラムの出力を調べ，1st:, 2nd:, 3rd: のそれぞれ後に出力される値を調べ，(1),(2),(3) として答えてください.

In [ ]:
```
a = 10

def plus_one(x):
    a = x+1

def plus_two(x):
    global a
    a = x+2

print( "1st:", a)
plus_one(3)
print( "2nd:", a)
plus_two(4)
print( "3rd:", a)
```

(1): XXX (2): XXX (3): XXX

課題提出

名前：

学籍番号：

感想：

ファイルを保存したら，このファイルをファイルメニューの「.ipynbをダウンロード」からダウンロードし，関大LMSで提出してください.

In [ ]:
```

```